

AI 데이터 분석 · 5일차

문자열 정리 메서드와 f-string

지저분한 글자를 다듬어, 사람이 읽는 리포트로

upper · lower · strip · replace · split · join · f-string

공백 제거 → 통일 · 치환 → 나누기 → 조각 정리 → 합치기 · 출력
교안 03_04(공백 · 대소문자) · 03_05(치환 · 분리 · 결합) · 03_06(f-string) 정리

2026-07-27 AI 데이터 분석 교육 정리 (5일차)
학습자 Minho · 국립금오공과대학교 컴퓨터공학부

이 책을 읽는 법

어제 문자열을 만들고·자르고·찾았다면, 오늘은 그 문자열을 “다듬습니다.” 대소문자·공백을 정리하고(메서드), 바꾸고 나누고 합치고(replace·split·join), 마지막에 f-string으로 사람이 읽는 문장으로 출력합니다. 뱃지로 “메서드 / 내장함수 / 문자열 / 연산자 / 개념” 을 먼저 구분하고 읽으면 됩니다.

메서드 s.upper()·s.strip() **문자열** f-string **내장함수** print·float **개념** 값=객체·정리 흐름

연결

오늘 배우는 정리 도구는 왜 필요할까요? 현장·산업 데이터는 대소문자·공백·표기가 제각각인 “지저분한 글자” 라, **분석·시각화 전에 반드시 정리(cleaning)**해야 같은 값이 따로 집계되지 않습니다. 오늘은 그 정리 도구 상자를 채웁니다.

목차

	장 / STEP	무엇을 배우나
CHAPTER 0	오늘의 지도 — 값은 객체, 메서드는 기능	점(.) 문법 · 원본은 안 바뀐다
STEP 1	대소문자 정리	upper·lower·title·isupper · 대소문자 무시 비교
STEP 2	공백 정리	strip·lstrip·rstrip·strip(문자) · 체이닝
STEP 3	치환·분리 — replace · split	바꾸기·제거 / 구분자로 나누기
STEP 4	결합 — join · sep	조각 합치기 / split↔join 짝꿍
STEP 5	f-string	{변수}·계산·소수점 :.2f
STEP 6	텍스트 정리 종합	5단계 흐름 · 센서 로그 리포트

CHAPTER 0

오늘의 지도 — 값은 객체, 메서드는 기능

점(.)으로 부르는 “값에 딸린 기능” 을 익히기 전에

오늘의 도구는 대부분 “메서드” 입니다. 메서드를 이해하려면 먼저 한 가지 개념 — 파이썬에서 값 하나하나가 “객체” 라는 것 — 을 잡아야 합니다.

개념 값은 객체, 메서드는 객체의 기능 점(.) 문법의 뿌리

정의 값 하나하나가 “객체” = 값 + 그 값에 딸린 기능입니다. 자료형은 객체의 종류이고(문자열엔 문자열 기능, 숫자엔 숫자 기능), 그 딸린 기능을 점(.)으로 부르는 것이 메서드입니다.

```
word = "python"
print(word.upper())          # PYTHON   (문자열 기능)
print(word.count("p"))       # 1
print(word.startswith("p")) # True
```

- ▶ PYTHON
- ▶ 1
- ▶ True

왜 쓰나 len()·int()는 혼자 부르는 내장 함수였지만(3일차), 오늘의 upper()·strip()은 “문자열에 붙어서” 부르는 메서드입니다. 그래서 항상 값.메서드() 형태예요.

개념

형태: 문자열.메서드이름(). 괄호를 꼭 붙여야 실행되고(word.upper는 실행 안 됨), 추가 정보가 필요하면 괄호 안에 인자를 넣습니다(count("p")). 인자가 없으면 괄호만 비웁니다.

개념 문자열 메서드는 원본을 바꾸지 않는다 가장 중요한 공통 규칙

정의 문자열 메서드는 원본을 그대로 두고 “새 결과” 를 돌려줍니다. 그 결과를 쓰려면 반드시 변수에 다시 받아야 합니다.

```
word = "python"
word.upper()          # 결과(PYTHON)를 안 받으면 사라짐
print(word)           # python   (원본 그대로!)
word = word.upper()   # 다시 받아야 반영
print(word)           # PYTHON
```

- ▶ python
- ▶ PYTHON

왜 쓰나 오늘 배울 `upper·lower·strip·replace·split·join` 전부 이 규칙을 따릅니다. “값이 안 바뀐 것 같으면 결과를 다시 받았는지 확인” 이 오늘의 1순위 디버깅 팁이에요. `word = word.upper()` 패턴을 손에 붙이세요.

오늘의 정리 흐름 미리보기

오늘 배우는 도구들은 결국 하나의 흐름으로 이어집니다. 지저분한 글자 한 줄을 이 다섯 걸음으로 깨끗하게 만듭니다.



STEP 1

대소문자 정리

같은 뜻인데 표기만 다른 데이터를 하나로

현장 데이터에는 같은 상태가 NORMAL·Normal·normal로 뒤섞여 있습니다. 통일하지 않으면 같은 상태가 따로 집계됩니다. 대소문자 메서드로 하나로 맞추습니다.

메서드 upper() · lower() 모두 대문자 / 소문자

정의 upper()는 영문을 모두 대문자로, lower()는 모두 소문자로 바꿉니다. 한글·숫자·기호는 그대로.

```
s = "ready"
print(s.upper())      # READY
print("WARNING".lower()) # warning
```

▶ READY
▶ warning

왜 쓰나 실무에서는 lower를 더 자주 씁니다 — 모두 소문자로 맞춰두면 비교·집계가 정확해지거든요. 결과는 반드시 새 변수에 받습니다(big = s.upper()).

메서드 capitalize() · title() 첫 글자만 대문자

정의 capitalize()는 문장 첫 글자만, title()은 단어마다 첫 글자를 대문자로 만듭니다.

```
print("python is".capitalize()) # Python is
print("kim chul soo".title())   # Kim Chul Soo
```

▶ Python is
▶ Kim Chul Soo

왜 쓰나 이름·제목 표기를 표준화할 때 가끔 씁니다. title은 “단어마다” 라 사람 이름 정리에 편해요.

메서드 대소문자 무시 비교 · isupper()·islower() lower로 맞춰 비교 / 검사

정의 비교 전에 양쪽을 lower()로 맞추면 대소문자와 무관하게 비교됩니다. isupper()·islower()는 모두 대/소문자인지 참·거짓으로 검사합니다(바꾸는 게 아니라 확인).

```
print("Fault" == "FAULT")          # False (표기 달라 다른 값)
print("Fault".lower() == "FAULT".lower())# True  (소문자로 통일 후)
print("ABC".isupper()) # True
print("Abc".isupper()) # False
```

- ▶ False
- ▶ True
- ▶ True
- ▶ False

왜 쓰나 “대소문자 무시 비교” 는 데이터를 다룰 때 표준처럼 쓰는 패턴입니다. is로 시작하는 메서드(isupper)는 대체로 “~인지 확인” 하는 검사용이에요.

설비 적용

파일명 규칙 점검: `fname.lower()`로 통일한 뒤 `startswith("sensor")·endswith(".csv")`로 한 번에 확인 — 통일 전 `"Sensor_LOG.CSV".endswith(".csv")`는 대문자라 False였습니다.

STEP 2

공백 정리

눈에 안 보이는 공백이 데이터를 망친다

"정상"과 "정상 "(뒤 공백)은 컴퓨터에겐 다른 값입니다. 같은 상태가 공백 차이로 따로 집계되죠. 그래서 데이터를 다루기 전 앞뒤 공백 제거는 거의 필수입니다.

메서드 strip() · lstrip()·rstrip() 앞뒤 / 한쪽 공백 제거

정의 strip()은 앞뒤 공백을 모두, lstrip()은 왼쪽(앞)만, rstrip()은 오른쪽(뒤)만 제거합니다.

```
text = " 정상 "  
print "[" + text + "]"          # [ 정상 ]  
print "[" + text.strip() + "]"  # [정상]  
print "[" + text.lstrip() + "]" # [정상]  
print "[" + text.rstrip() + "]" # [ 정상]
```

```
▶ [ 정상 ]  
▶ [정상]  
▶ [정상]  
▶ [ 정상]
```

왜 쓰나 양옆에 []를 붙여 출력하면 공백이 사라진 걸 눈으로 확인할 수 있습니다. 보통은 strip 하나로 양쪽 다 떼면 충분하고, 한쪽만 살릴 때만 l/rstrip을 씁니다.

자주 하는 실수

결과를 안 받으면 원본 그대로입니다. text.strip()만 쓰고 끝내면 정리 결과가 사라져요 → text = text.strip()으로 다시 받으세요(가장 흔한 실수).

메서드 strip("문자") 특정 문자 제거

정의 괄호에 글자를 넣으면 양 끝에서 그 글자를 (나올 때까지 계속) 제거합니다. 가운데 글자는 건드리지 않습니다.

```
print("***경고***".strip("*")) # 경고  
print("###정상###".strip("#")) # 정상
```

```
▶ 경고  
▶ 정상
```

왜 쓰나 라벨을 감싼 기호(*, #)를 벗겨낼 때 편합니다. 단, strip은 “앞뒤 전용” — 글자 사이 공백은 못 지웁니다("정상".strip()은 "정상"). 중간 공백은 STEP 3의 replace로 해결합니다.

메서드 체이닝 strip().lower() 정리 단계를 한 줄로

정의 메서드 뒤에 또 메서드를 점으로 이어 붙입니다. 왼쪽에서 오른쪽 순서로 적용됩니다.

```
raw = " NORMAL "  
print(raw.strip())          # NORMAL  
print(raw.strip().lower())  # normal (공백 제거 후 소문자)
```

```
▶ NORMAL  
▶ normal
```

왜 쓰나 입력값은 보통 strip 후 lower 순으로 정리합니다. 정리 단계가 여러 개일 때 체이닝으로 한 줄에 깔끔하게 처리할 수 있어요. “입력이 들어오면 일단 정리부터” 가 좋은 습관입니다.

STEP 3

치환·분리 — replace · split

바꾸고 지우고, 구분자로 나누기

메서드 replace(A, B) 바꾸기 · 제거

정의 A를 B로 바꿉니다(등장하는 걸 모두 한 번에). 빈 값 ""으로 바꾸면 그 글자가 사라집니다.

```
print("설비 정상 가동".replace("정상", "점검")) # 설비 점검 가동
print("정 상 가 동".replace(" ", ""))          # 정상가동 (중간 공백까지)
print("010-1234-5678".replace("-", ""))        # 01012345678
```

- ▶ 설비 점검 가동
- ▶ 정상가동
- ▶ 01012345678

왜 쓰나 표기 통일(fault·FAULT → 고장)과 기호 제거(3,000의 쉼표, 전화번호 하이픈)에 씁니다. strip이 못 하던 “중간 공백” 도 replace(' ', '')로 해결돼요. 체이닝도 됩니다: .replace("-", "").replace(" ", "").

메서드 split() · split(",") 구분자로 나누기 (→ 리스트)

정의 정해진 구분자로 문자열을 여러 조각으로 나눠 리스트로 돌려줍니다. 괄호를 비우면 공백 기준, 넣으면 그 구분자 기준.

```
print("PUMP A 03".split())      # ['PUMP', 'A', '03'] (공백 기준)
print("a,b,c,d".split(","))     # ['a', 'b', 'c', 'd']
print("2025-01-15".split("-")) # ['2025', '01', '15']
print("a,b,c,d".split(",", 1)) # ['a', 'b,c,d'] (1번만 나눔)
```

- ▶ ['PUMP', 'A', '03']
- ▶ ['a', 'b', 'c', 'd']
- ▶ ['2025', '01', '15']
- ▶ ['a', 'b,c,d']

왜 쓰나 슬라이싱은 “위치로” 자르고, split은 “구분자로” 나눕니다. CSV 한 줄("1,EQP-001,정상,25.3")을 쉼표로 나눠 각 값을 꺼낼 때 핵심이에요. 나온 조각은 리스트라 번호로 꺼냅니다(.split("-")[0]이 연도).

개념

리스트: 여러 값을 순서대로 담은 그릇(대괄호 []). 0번부터 번호로 조각을 꺼내며, 자세한 건 다음 단원 — 지금은 split 결과를 이해하는 용도로 봅니다.

STEP 4

결합 — join · sep

나눈 조각을 다시 하나로

메서드 `join()` 조각을 하나의 문자열로

정의 리스트의 여러 조각을 하나의 문자열로 합칩니다. 순서가 독특해 “구분자”가 앞에 옵니다 —

구분자.`join(리스트)`.

```
parts = ["2025", "01", "15"]
print("-".join(parts))    # 2025-01-15
print("").join(parts)    # 20250115 (딱 붙임)
print(" / ".join(parts)) # 2025 / 01 / 15
```

```
▶ 2025-01-15
▶ 20250115
▶ 2025 / 01 / 15
```

왜 쓰나 `split()`로 나눈 데이터를 다시 합칠 때 씁니다. 앞의 구분자에 따라 합쳐진 모양이 달라져요.

자주 하는 실수

`join` 대상은 “문자열 조각”만 됩니다. 숫자가 섞이면 오류 → `str()`로 먼저 문자열로 바꾸세요.

개념 `split` ↔ `join` 은 짝꿍 · `print`의 `sep` 구분자 통째로 교체

정의 `split`은 문자열→리스트, `join`은 리스트→문자열로 정반대입니다. 둘을 묶으면 구분자를 통째로 바꿀 수 있습니다.

`print`의 `sep`은 출력할 때 값들 사이에 넣는 구분자.

```
raw = "2026/07/27"
parts = raw.split("/")    # ['2026', '07', '27']
print("-".join(parts))    # 2026-07-27 ( / 를 - 로 교체)
print(2026, 7, 27, sep="-") # 2026-7-27 (출력 시 구분자)
```

```
▶ 2026-07-27
▶ 2026-7-27
```

왜 쓰나 `split()`으로 나누고 다른 구분자로 `join()`하면 “구분자 통째 교체”가 됩니다(`/` → `-`). `join`은 리스트를 문자열로 합치는 것이고, `sep`은 `print`가 출력할 때 사이에 끼우는 것 — 목적이 다릅니다.

한 결과, 네 가지 방법 — "python" → "pyThon"

실습에서 t만 대문자로 바꿔 pyThon을 만드는 방법이 여러 가지였습니다. 오늘 배운 도구들이 어떻게 조합되는지 한눈에 보여줍니다.

```
python = "python"
# 방법1 인덱싱+슬라이싱 python[0:2] + python[2].upper() + python[3:]
# 방법2 replace          python.replace("t", "T")
# 방법3 글자별 조합      python[0]+...+python[2].upper()+...
# 방법4 split + join      "T".join(python.split("t"))
```

▶ pyThon (네 방법 모두 같은 결과)

STEP 5

f-string

숫자·글자 섞인 문장을 깔끔하게 출력

숫자·글자를 섞은 문장을 +로 잇는 건 번거롭습니다(따옴표·더하기 남발, 숫자마다 str). 이 불편을 푸는 도구가 f-string 입니다.

문자열 f-string `f"...{변수}..."` 문장에 값 바로 끼우기

정의 따옴표 앞에 f를 붙이고, 값을 넣을 자리에 중괄호 {변수}를 씁니다. f가 “중괄호를 변수로 해석하라”는 신호예요.

```
name = "홍길동"
age = 25
print(f"{name}님은 {age}살입니다")    # 홍길동님은 25살입니다
code = "EQP-001"
print(f"설비 {code} 점검 완료")      # 설비 EQP-001 점검 완료
```

▶ 홍길동님은 25살입니다
▶ 설비 EQP-001 점검 완료

왜 쓰나 중괄호 안에는 숫자를 str로 안 바꿔도 그냥 끼워집니다(형변환 자동). 완성될 문장을 그대로 쓰고 변수만 중괄호로 표시하니 한눈에 읽혀요.

자주 하는 실수

가장 흔한 실수 두 가지: ① 따옴표 앞 f를 빠뜨림(중괄호와 변수 이름이 그대로 출력됨) ② 중괄호 안에 따옴표를 넣음. 이상하면 “따옴표 바로 앞에 f가 있나” 부터 확인.

문자열 f-string 안에서 계산 · 소수점 `{:.2f}` 식·자릿수 지정

정의 중괄호 안에 계산식을 넣으면 결과가 들어갑니다. `{값:.2f}`처럼 콜론 뒤 .2f를 붙이면 소수점 자릿수를 지정(자동 반올림)합니다.

```
a, b, c = 80, 91, 90
print(f"평균 {(a + b + c) / 3}")    # 평균 87.0
value = 25.34567
print(f"측정값 {value:.2f}")      # 측정값 25.35
print(f"측정값 {value:.1f}")      # 측정값 25.3
```

- ▶ 평균 87.0
- ▶ 측정값 25.35
- ▶ 측정값 25.3

왜 쓰나 측정값·평균·비율을 보고용으로 정리할 때 :.2f가 특히 유용합니다. 분석의 마지막은 결국 숫자를 “사람이 읽는 문장”으로 바꾸는 일이고, 그게 우리 과정의 최종 산출물인 정비 판단 문장이예요.

+ 연결 vs f-string

같은 문장을 만들 때 f-string이 훨씬 짧고 안전합니다.

방식	같은 문장 "온도: 87도" 만들기
+ 연결	"온도: " + str(temp) + "도"
f-string	f"온도: {temp}도"

개념

더하기는 따옴표·더하기가 많고 숫자마다 str()이 필요하지만, f-string은 종괄호로 끝나고 실수가 적습니다. 실무에서 문장 만들 땐 거의 항상 f-string이예요.

STEP 6

텍스트 정리 종합

지저분한 한 줄 → 깨끗한 리포트

오늘 배운 도구를 모으면, 어떤 지저분한 데이터를 만나도 당황하지 않는 흐름이 됩니다. 정리는 보통 아래 순서로 흐릅니다(필요한 단계만 골라 씀).



종합 실습 — 센서 로그 한 줄 정리 리포트

앞뒤 공백·대소문자·구분자가 뒤섞인 로그 한 줄을 정리해, f-string 한 줄 판단 리포트로 출력합니다. 오늘의 모든 도구가 한 번에 들어갑니다.

```
raw = " 5 , sensor_2 , WARNING , 0.78912 "
parts = raw.strip().split(",")          # 앞뒤 공백 떼고 쉼표로 분리
sid   = parts[1].strip()                # sensor_2
status = parts[2].strip().lower()       # warning
value = float(parts[3].strip())         # 0.78912
print(f"[센서 {sid}] 상태 {status}, 측정값 {value:.2f}")
```

▶ [센서 sensor_2] 상태 warning, 측정값 0.79

설비 적용

현장 사례가 다 이 패턴입니다 — **이메일**은 strip+lower, **전화번호**는 replace로 하이픈 제거, **이름**은 strip+title, **컬럼명**은 lower+replace(공백→밑줄). 모두 “정리 → f-string 출력” 으로 마무리됩니다.

부록 A

텍스트 도구 총정리 치트시트

문자열 4일치를 한 장에

3~5일차에 배운 문자열 도구를 목적별로 모았습니다. 시험·복습 전에 이 표만 훑으면 됩니다.

목적	도구
만들기·출력	따옴표"..." · print() · f-string f"{x}"
형변환	str() · int() · float()
꺼내기	인덱싱 s[0] · 슬라이싱 s[1:4] · split()
확인·검색	len() · in · count() · find() · startswith()
다듬기	strip · lower/upper · replace · split · join

오늘의 메서드 요약

메서드	기능	예시
upper/lower	대문자 / 소문자	"a".upper() → A
strip()	앞뒤 공백 제거	" a ".strip() → a
replace(A,B)	바꾸기·제거	"a-b".replace("-", "") → ab
split(",")	구분자로 나누기(→리스트)	"a,b".split(",") → [a,b]
"-".join(x)	조각 합치기(→문자열)	"-".join([a,b]) → a-b
f"{v:.2f}"	문장에 값·소수점	f"{3.14159:.2f}" → 3.14

부록 B

오류·함정 사전

오늘 코드에서 걸려 넘어지기 쉬운 곳

오늘 함정은 대부분 “결과를 안 받았거나, 종류가 안 맞아서” 생깁니다. 왼쪽처럼 하면 오른쪽으로 고치세요.

X 이렇게 하면	무슨 일	✓ 바로잡기
<code>text.strip()</code> (만 쓰고 끝)	결과 사라짐, 원본 그대로	<code>text = text.strip()</code>
<code>" 정상 ".strip()</code>	가운데 공백은 안 지워짐	<code>.replace(" ", "")</code>
<code>"Fault"=="FAULT"</code>	False (표기 다름)	<code>.lower()==.lower()</code>
<code>"-".join([2025,1])</code>	오류 (숫자 섞임)	str로 먼저 변환
<code>f 빠뜨림 "{x}"</code>	중괄호 그대로 출력	<code>f"{x}"</code>
<code>word.upper</code> (괄호 X)	실행 안 됨(함수 자체)	<code>word.upper()</code>

주의

오늘의 1순위 디버깅 팁: “값이 안 바뀐 것 같다” → **메서드 결과를 변수에 다시 받았는지** 먼저 확인하세요.
문자열 메서드는 원본을 절대 바꾸지 않습니다.

맺음말

오늘의 성취와 다음 한 걸음

지저분한 글자 → 깨끗한 리포트, 한 바퀴 완성

오늘로 문자열 다루기 4부작이 끝났습니다 — 만들고(생성), 자르고(인덱싱·슬라이싱), 찾고(len·in·find), 오늘 다듬어(strip·lower·replace·split·join) f-string으로 리포트까지. 이제 현장에서 마주치는 지저분한 텍스트 한 줄을 분석 가능한 형태로 정리할 수 있게 됐습니다. 이것이 데이터 분석·시각화로 넘어가기 전의 마지막 기본기입니다.

개념

복습 루틴 추천: ① `03\_05\_example.py`의 “pyThon 만들기 4가지 방법”을 직접 쳐보며 도구 조합 감각 익히기 → ② `" NORMAL "` 같은 지저분한 값에 strip→lower→replace를 한 줄 체이닝으로 정리 → ③ 센서 로그 한 줄을 split→strip→f-string으로 리포트 만들기 → ④ “결과를 다시 받았는지”를 항상 확인. 다음 단원은 오늘 살짝 맛본 리스트, 그리고 조건문(if)으로 “정리한 값에 따라 판단”하는 단계로 이어집니다.

— 끝 —